Lab: Set Up Unity Catalog on Azure Databricks using Terraform

Description

In this lab, you will set up Unity Catalog, the data governance layer of Azure Databricks, entirely with Terraform. You will use the AzureRM provider (v5.6.0) and the Databricks provider (v1.132.0).

The lab has two parts:

- Part A (Azure infrastructure): an ADLS Gen2 storage account, a Databricks Access Connector (a managed identity), and the permissions that let the connector read and write the storage.
- Part B (data governance): the Unity Catalog metastore, a storage credential, external locations, a catalog with bronze, silver and gold schemas, and grants that control who can use them.

This lab bridges the gap between platform engineers who build the infrastructure and data engineers who work with the data. By the end, you will create a governed table on your own storage and query it with SQL.

Learning Objectives

By the end of this lab, you will be able to:

1. Create an ADLS Gen2 storage account with containers for managed and raw data.
2. Create a Databricks Access Connector and grant it Storage Blob Data Contributor on the storage account.
3. Configure the Databricks Terraform provider against a workspace created in the same project.
4. Use an existing Unity Catalog metastore, or create one and assign it to the workspace.
5. Define a storage credential and external locations that point to your storage.
6. Create a catalog with managed storage and bronze, silver and gold schemas.
7. Apply grants that control who can use the catalog, schemas and storage paths.
8. Load a CSV file into a governed table and verify where Unity Catalog stores it.

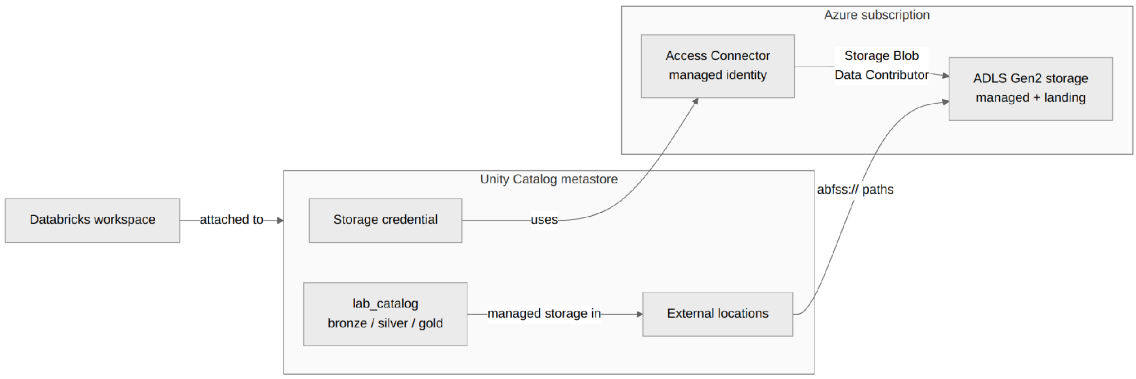
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Owner**, or **Contributor** plus **User Access Administrator**. You need permission to create role assignments.
- **Terraform** and **Azure CLI** installed locally.
- **Visual Studio Code (VS Code)** or any IDE.
- Completion of the earlier lab, *Provision an Azure Databricks Workspace using Terraform*.

*Unity Catalog requires the **Premium** pricing tier.*

Architecture



The **Access Connector** is the identity Databricks uses to reach your storage. Unity Catalog wraps it in a **storage credential**, and **external locations** combine that credential with specific storage paths. The catalog stores its managed tables inside one of those locations.

Key Unity Catalog Objects

Object	What it is	Created by
Metastore	Top-level container for all Unity Catalog metadata. One per region per Databricks account.	Databricks (automatic) or Terraform

Object	What it is	Created by
Access Connector	Azure managed identity that Databricks uses to access storage	AzureRM provider
Storage credential	Unity Catalog object that wraps the Access Connector	Databricks provider
External location	A storage path plus a credential. Controls who can read and write that path.	Databricks provider
Catalog	First level of the three-level namespace: catalog.schema.table	Databricks provider
Schema	Second level. Groups tables, views and volumes.	Databricks provider

Most Azure Databricks accounts already have a metastore in each region. New workspaces are attached to it automatically. Step 9 covers both reusing it and creating a new one.

Step by Step Guide

Part A: Provision the Azure Infrastructure

Step 1: Authenticate to Azure

```
az login
az account set --subscription "<your-subscription-id>"
```

Step 2: Create the Project and Configure the Providers

Create a new folder called **databricks_unity_catalog**. Inside it, create a file named **providers.tf**:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
```

```
    version = "5.6.0"
   }

   random = {
    source  = "hashicorp/random"
    version = "~> 3.7"
   }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your-subscription-id>"

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.Storage",
  ]
}
```

Explanation:

- random → generates a unique suffix, because storage account names must be unique across all of Azure.

Initialize the project:

```
terraform init
```

Step 3: Define Variables and Values

Create a file named variables.tf:

```
variable "resource_group_name" {
  type = string
}

variable "location" {
  type    = string
```

```
  default = "West Europe"
}

variable "workspace_name" {
  type = string
}

variable "managed_resource_group_name" {
  type = string
}

variable "access_connector_name" {
  type = string
}

variable "catalog_name" {
  type    = string
  default = "lab_catalog"
}

variable "tags" {
  type = map(string)
}
```

Create a file named terraform.tfvars:

```
resource_group_name         = "rg-databricks-uc-lab"
location                    = "West Europe"
workspace_name              = "dbw-uc-lab"
managed_resource_group_name = "rg-databricks-uc-lab-managed"
access_connector_name       = "ac-databricks-uc-lab"
catalog_name                = "lab_catalog"
tags = {
  environment = "dev"
  project     = "databricks-unity-catalog-lab"
}
```

Step 4: Create the Resource Group and Workspace

Create a file named workspace.tf:

```
resource "azurerm_resource_group" "uc" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

resource "azurerm_databricks_workspace" "workspace" {
  name                        = var.workspace_name
  resource_group_name         = azurerm_resource_group.uc.name
  location                    = azurerm_resource_group.uc.location
  sku                         = "premium"
  managed_resource_group_name = var.managed_resource_group_name
  tags                        = var.tags
}
```

This lab uses a simple workspace to keep the focus on Unity Catalog. The same steps work with the VNet injection, Private Link and hub-and-spoke workspaces from earlier labs.

Step 5: Create the ADLS Gen2 Storage Account

Create a file named storage.tf:

```
resource "random_string" "suffix" {
  length  = 6
  special = false
  upper   = false
}

resource "azurerm_storage_account" "uc" {
  name                     = "stuclab${random_string.suffix.result}"
  resource_group_name      = azurerm_resource_group.uc.name
  location                 = azurerm_resource_group.uc.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
```

```
  is_hns_enabled           = true
  min_tls_version          = "TLS1_2"
  tags                     = var.tags
}

resource "azurerm_storage_container" "uc" {
  for_each = toset(["managed", "landing"])

  name                  = each.key
  storage_account_id    = azurerm_storage_account.uc.id
  container_access_type = "private"
}
```

Explanation:

- is_hns_enabled = true → turns on the hierarchical namespace, which is what makes this an ADLS Gen2 account with real folders.
- managed container → where Unity Catalog stores the files for managed tables in your catalog.
- landing container → where raw files arrive before they are loaded into tables.
- storage_account_id → from AzureRM v5.0, containers reference the account by ID instead of name.

Step 6: Create the Access Connector and Grant Storage Access

Create a file named access_connector.tf:

```
resource "azurerm_databricks_access_connector" "uc" {
  name                = var.access_connector_name
  resource_group_name = azurerm_resource_group.uc.name
  location            = azurerm_resource_group.uc.location

  identity {
    type = "SystemAssigned"
  }

  tags = var.tags
}
```

```
resource "azurerm_role_assignment" "connector_storage" {
  scope                = azurerm_storage_account.uc.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_databricks_access_connector.uc.identity[0].principal_id
}

data "azurerm_client_config" "current" {}

resource "azurerm_role_assignment" "me_storage" {
  scope                = azurerm_storage_account.uc.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = data.azurerm_client_config.current.object_id
}
```

Explanation:

- SystemAssigned identity → Azure creates and manages a Microsoft Entra ID identity for the connector. There are no secrets to rotate.
- connector_storage → lets Databricks read and write data in the storage account through the connector.
- me_storage → lets you upload a test file in Step 15. It is not needed by Unity Catalog.

Step 7: Define Outputs and Deploy Part A

Create a file named outputs.tf:

```
output "databricks_workspace_url" {
  value = "https://${azurerm_databricks_workspace.workspace.workspace_url}"
}

output "storage_account_name" {
  value = azurerm_storage_account.uc.name
}

output "access_connector_id" {
  value = azurerm_databricks_access_connector.uc.id
}
```

Deploy:

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

Part A takes about 5 minutes. Role assignments can take a few more minutes to take effect, and Part B gives them that time.

Part B: Configure Unity Catalog

Step 8: Add the Databricks Provider

The workspace now exists, so Terraform can connect to it. Replace the contents of **providers.tf** with:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "5.6.0"
    }

    random = {
      source  = "hashicorp/random"
      version = "~> 3.7"
    }

    databricks = {
      source  = "databricks/databricks"
      version = "1.132.0"
    }
  }
}

provider "azurerm" {
  features {}
```

```
  subscription_id = "<your-subscription-id>"

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.Storage",
  ]
}

provider "databricks" {
  host                        = azurerm_databricks_workspace.workspace.workspace_url
  azure_workspace_resource_id = azurerm_databricks_workspace.workspace.id
}
```

Download the new provider:

```
terraform init -upgrade
```

Explanation:

- Two-part deployment → the Databricks provider needs a running workspace to connect to. Creating the workspace in Part A first avoids connection errors during planning.
- Authentication → the provider reuses your az login session.

Step 9: Get the Metastore

Choose one of the two options below.

Option A (default): Reuse the existing regional metastore

Most accounts already have a metastore in the region, and the new workspace is attached to it automatically. Create a file named metastore.tf:

```
data "databricks_current_metastore" "this" {}

locals {
  metastore_id = data.databricks_current_metastore.this.id
}
```

Option B: Create and assign a new metastore

Use this only if no metastore exists in your region and you are a Databricks account admin. A region can have only one metastore per account, so this fails if one already exists.

Find your account ID in the account console at accounts.azuredatabricks.net (user menu, top right). Then create metastore.tf with:

```
variable "databricks_account_id" {
  type = string
}

provider "databricks" {
  alias      = "account"
  host       = "https://accounts.azuredatabricks.net"
  account_id = var.databricks_account_id
}

resource "databricks_metastore" "this" {
  provider      = databricks.account
  name          = "metastore-${azurerm_databricks_workspace.workspace.location}"
  region        = azurerm_databricks_workspace.workspace.location
  force_destroy = true
}

resource "databricks_metastore_assignment" "this" {
  provider     = databricks.account
  metastore_id = databricks_metastore.this.id
  workspace_id = azurerm_databricks_workspace.workspace.workspace_id
}

locals {
  metastore_id = databricks_metastore_assignment.this.metastore_id
}
```

Add your account ID to terraform.tfvars:

```
databricks_account_id = "<your-databricks-account-id>"
```

Explanation:

- Account-level provider → metastores belong to the Databricks account, not a workspace, so they are managed through accounts.azuredatabricks.net.

- No storage root → the metastore has no default storage. Each catalog defines its own managed storage instead, which is the current best practice.
- locals.metastore_id → both options expose the same value, so the rest of the lab works unchanged.

Step 10: Create the Storage Credential

Create a file named storage_credential.tf:

```
resource "databricks_storage_credential" "adls" {
  name         = "cred-${var.access_connector_name}"
  metastore_id = local.metastore_id
  comment      = "Access Connector for the Unity Catalog lab"

  azure_managed_identity {
    access_connector_id = azurerm_databricks_access_connector.uc.id
  }

  force_destroy = true
}
```

Explanation:

- azure_managed_identity → registers the Access Connector with Unity Catalog. No keys or secrets are stored anywhere.

Step 11: Create the External Locations

Create a file named external_locations.tf:

```
locals {
  storage_host = "${azurerm_storage_account.uc.name}.dfs.core.windows.net"
}

resource "databricks_external_location" "this" {
  for_each = azurerm_storage_container.uc

  name            = "loc-${each.key}"
  url             = "abfss://${each.value.name}@${local.storage_host}/"
```

```
  credential_name = databricks_storage_credential.adls.name
  comment         = "The ${each.key} container in the lab storage account"
  force_destroy   = true
}
```

Explanation:

- abfss:// → the secure ADLS Gen2 path format: abfss://container@account.dfs.core.windows.net/path.
- for_each → creates one external location per container: loc-managed and loc-landing.
- Unity Catalog checks that the credential can reach the path when you create the location. If the role assignment from Step 6 is missing, this step fails.

Step 12: Create the Catalog and Schemas

Create a file named catalog.tf:

```
resource "databricks_catalog" "lab" {
  name          = var.catalog_name
  storage_root  = "${databricks_external_location.this["managed"].url}${var.catalog_name}"
  comment       = "Catalog for the Unity Catalog lab"
  force_destroy = true

  properties = {
    purpose = "terraform-lab"
  }
}

resource "databricks_schema" "layers" {
  for_each = {
    bronze = "Raw data as ingested"
    silver = "Cleaned and conformed data"
    gold   = "Business-level aggregates"
  }

  catalog_name  = databricks_catalog.lab.name
  name          = each.key
  comment       = each.value
```

```
  force_destroy = true
}
```

Explanation:

- storage_root → managed tables in this catalog store their files under managed/lab_catalog in your own storage account.
- Bronze, silver, gold → the medallion architecture: raw data, then cleaned data, then business-ready data.
- force_destroy → lets terraform destroy remove the catalog and schemas even if they contain tables. Leave it out in production.

Step 13: Grant Access

Create a file named grants.tf:

```
resource "databricks_grants" "catalog" {
  catalog = databricks_catalog.lab.name

  grant {
    principal  = "account users"
    privileges = ["USE_CATALOG", "USE_SCHEMA", "SELECT"]
  }
}

resource "databricks_grants" "bronze" {
  schema = databricks_schema.layers["bronze"].id

  grant {
    principal  = "account users"
    privileges = ["CREATE_TABLE", "MODIFY"]
  }
}

resource "databricks_grants" "landing" {
  external_location = databricks_external_location.this["landing"].id

  grant {
```

```
    principal  = "account users"
    privileges = ["READ_FILES"]
  }
}
```

Explanation:

- Catalog grant → every user can browse the catalog and read any table in it. Privileges granted on a catalog are inherited by all its schemas and tables.
- Bronze grant → users can create and write tables in bronze only. Silver and gold stay read-only.
- Landing grant → users can read raw files from the landing container. Nobody gets direct file access to the managed container.
- databricks_grants is authoritative → it replaces all grants on the object with exactly what is in the code. Changes made in the UI are removed on the next apply.

In production, grant to groups such as data_engineers or analysts instead of account users.

Step 14: Add Outputs and Deploy Part B

Add these to outputs.tf:

```
output "metastore_id" {
  value = local.metastore_id
}

output "catalog_name" {
  value = databricks_catalog.lab.name
}

output "landing_path" {
  value = databricks_external_location.this["landing"].url
}
```

Deploy:

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

Step 15: Verify Unity Catalog

15.1 Explore the objects in Catalog Explorer

1. Open the **databricks_workspace_url** and click **Catalog** in the left menu.
2. Expand **lab_catalog**. You see the **bronze**, **silver** and **gold** schemas.
3. Open the **Permissions** tab on **lab_catalog**. **account users** has **USE CATALOG**, **USE SCHEMA** and **SELECT**.
4. Click the gear icon, then **External data**. Under **Credentials**, you see your storage credential. Under **External locations**, you see **loc-managed** and **loc-landing**.
5. Open **loc-landing** and click **Test connection**. All checks pass.

15.2 Upload a test file

In your local terminal, inside the project folder, create a small CSV file and upload it to the landing container:

```
cat > customers.csv << 'EOF'
customer_id,name,city
1,Asha,Mumbai
2,Ben,London
3,Chen,Singapore
EOF

az storage blob upload \
  --account-name $(terraform output -raw storage_account_name) \
  --container-name landing \
  --name customers/customers.csv \
  --file customers.csv \
  --auth-mode login
```

15.3 Load and query the data with SQL

1. In the workspace, open **SQL Editor**.
2. Select the **Serverless Starter Warehouse**. If it does not exist, create a **Small** SQL warehouse with **Auto stop** set to **10 minutes**.
3. Run the queries below. Replace **stuclabxxxxxx** with your **storage_account_name** output.

```
-- List the raw files through the external location
LIST 'abfss://landing@stuclabxxxxxx.dfs.core.windows.net/customers/';

-- Load the CSV into a managed table in the bronze schema
CREATE TABLE lab_catalog.bronze.customers AS
SELECT *
FROM read_files(
  'abfss://landing@stuclabxxxxxx.dfs.core.windows.net/customers/',
  format => 'csv',
  header => true
);

-- Query the table
SELECT * FROM lab_catalog.bronze.customers;

-- See where Unity Catalog stored the table files
DESCRIBE DETAIL lab_catalog.bronze.customers;

-- Review the permissions
SHOW GRANTS ON SCHEMA lab_catalog.bronze;
```

Query	What you should see
LIST	customers.csv in the landing container
SELECT	Three rows: Asha, Ben and Chen
DESCRIBE DETAIL	A location starting with abfss://managed@stuclabxxxxxx.dfs.core.windows.net/lab_catalog/
SHOW GRANTS	CREATE TABLE and MODIFY for account users, plus inherited catalog privileges

15.4 See the governance in action

1. In Catalog Explorer, open lab_catalog → bronze → customers and select the Lineage tab. The table shows the landing file path as its upstream source.

2. In the Azure Portal, open your storage account, then the managed container. Under lab_catalog, you see the __unitystorage folder holding the table files. The data lives in your storage, while Unity Catalog controls who can access it.

Step 16: Clean Up Resources

Destroy everything:

```
terraform destroy --auto-approve
```

Terraform removes the Unity Catalog objects first, then the Azure resources. With Option A, the shared regional metastore is not deleted, because Terraform only read it. With Option B, the metastore you created is deleted. Delete customers.csv from your machine.

Troubleshooting

- Databricks provider authentication error during plan → make sure Part A was applied first and your az login session is still valid.
- Option A: no metastore assigned to the workspace → your region has no metastore. Use Option B, or ask a Databricks account admin to create one.
- Option B: a metastore already exists in this region → use Option A instead.
- External location fails with a permission error → the role assignment for the Access Connector may not be active yet. Wait five minutes and run terraform apply again.
- Catalog storage root is not covered by an external location → the storage_root must sit under the loc-managed URL.
- Role assignment fails with AuthorizationFailed → you need Owner or User Access Administrator on the subscription or resource group.
- az storage blob upload fails with AuthorizationPermissionMismatch → your own role assignment is still propagating. Wait a few minutes and retry.

Outcomes

You've set up Unity Catalog on Azure Databricks end to end with Terraform. You now know how to:

- Provision ADLS Gen2 storage and a Databricks Access Connector with least-privilege role assignments.

- Use the AzureRM and Databricks providers together in one project, deployed in two parts.
- Reuse the regional metastore, or create one and assign it to a workspace at account level.
- Register a storage credential and external locations that point to your own storage.
- Build a catalog with managed storage and bronze, silver and gold schemas.
- Control access with inherited grants at catalog, schema and external location level.
- Load raw files into a governed table and trace them with lineage.

This sets the foundation for group-based access control, SQL warehouses and jobs managed as code, and sharing data across workspaces with Delta Sharing.